PES-VCS Lab Report

Version Control System

NAME: YASHAS K C | SRN: PES2UG24CS669

Phase 1: Object Storage

Screenshot 1A — ./test_objects (All Tests Passing)

Command: ./test_objects

```
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$
```

All Phase 1 tests passed: blob storage, deduplication, and integrity check. The object was stored at the correct sharded path derived from its SHA-256 hash.

Screenshot 1B — find .pes/objects -type f (Sharded Directory Structure)

Command: find .pes/objects -type f

```
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$
```

Three blob objects are visible, each stored under a two-character prefix directory (e.g., d5/..., 2a/..., 25/...). This mirrors Git's fan-out structure to avoid large flat directories.

Phase 2: Tree Objects

Screenshot 2A — ./test_tree (All Tests Passing)

Command: ./test_tree

```
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$
```

All Phase 2 tests passed: tree serialize/parse roundtrip and deterministic serialization both succeeded. Serialized tree size was 139 bytes.

Screenshot 2B — xxd of Raw Tree Object (First 20 Lines)

Command: xxd .pes/objects/3a/3e22c686474097b70bc8a8e46e13e1046fbcd... | head -20

```
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ xxd .pes/objects/3a/3e22c686474097b70bc8a8e46e13e1046fbcd
c09fac7796143ecbfb422f05c | head -20
00000000: 7472 6565 2034 3900 3130 3036 3634 2066  tree 49.100664 f
00000010: 696c 6531 2e74 7874 002c f8d8 3d9e e295  ile1.txt.,..=...
00000020: 43b3 4a87 7274 21fd ecb7 e3f3 a183 d337  C.J.rt!........7
00000030: 6390 25de 576d b9eb b4                   c.%.Wm...
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$
```

The raw binary tree object begins with the header 'tree 49' followed by a null byte, then binary-encoded entries. Each entry encodes the file mode (e.g., 100644), filename, and the 20-byte binary SHA-256 hash of the referenced blob, confirming correct serialization format.

Phase 3: Staging Area (Index)

Screenshot 3A — pes init -> pes add -> pes status

Commands: ./pes init | ./pes add file1.txt file2.txt | ./pes status

```
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ ./pes add file1.txt file2.txt
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_tree
  untracked:  test_sequence.sh
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  Makefile
  untracked:  README.md
  untracked:  index.h
  untracked:  pes.h
  untracked:  tree.h
  untracked:  object.c
  untracked:  index.c
  untracked:  .gitignore
  untracked:  test_tree.c
  untracked:  test_objects
  untracked:  commit.h
  untracked:  tree.c
  untracked:  pes.c
```

The repository was successfully initialized in .pes/. Both file1.txt and file2.txt appear under 'Staged changes'. Unstaged changes show nothing, confirming the index was correctly updated.

Screenshot 3B — cat .pes/index (Text-Format Index)

Command: cat .pes/index

```
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776802733 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776800665 6 file2.txt
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$
```

The index stores two entries in the format: <mode> <hash> <mtime> <size> <path>. Both file1.txt and file2.txt have mode 100644, their respective SHA-256 blob hashes, modification timestamps, and sizes (6 bytes each).

Phase 4: Commits and History

Screenshot 4A — pes log (Three Commits)

Command: ./pes log

```
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ ./pes log
commit 702451e82b7cfbb0df8abe8cef9b80c4442129c6f939a3cabc36a8cf086bced4
Author: YASHAS K C <PES2UG24CS669>
Date:   1776803333

    third commit◆◆

commit 4fcffc1baffda3c8f01374c4b67ab7b0bda9e9059b3d19ed3f6fd62d7203f4e0
Author: YASHAS K C <PES2UG24CS669>
Date:   1776803304

    second commit

commit c2dd5b0138fb03a0aa1bbd6e4bcff8b0f7667dbfd7812ca96c7ea93aaad68edd
Author: YASHAS K C <PES2UG24CS669>
Date:   1776802754

    testa
```

The log shows three commits in reverse-chronological order: 'third commit', 'second commit', and 'testa'. Each entry includes the full commit hash, author (YASHAS K C <PES2UG24CS669>), Unix timestamp, and message.

Screenshot 4B — find .pes -type f | sort (Object Store Growth)

Command: find .pes -type f | sort

```
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/02/ac00490d94dddfb432b07ebb4766c8471992fee8d55f653bbbb5113ddebd7d
.pes/objects/19/49f61f07d0d020cb1e98ef105d7c0584d6c811f58fbda7d41925bee295e495
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/3a/3e22c686474097b70bc8a8e46e13e1046fbcdc09fac7796143ecbfb422f05c
.pes/objects/45/c3e4d7c0caf5a176b51e38ed53f72acdd648bdb4a25e66b50a2120479fb3de
.pes/objects/4f/cffc1baffda3c8f01374c4b67ab7b0bda9e9059b3d19ed3f6fd62d7203f4e0
.pes/objects/70/2451e82b7cfbb0df8abe8cef9b80c4442129c6f939a3cabc36a8cf086bced4
.pes/objects/7a/0068c1779d8e7354c9c3b0b285d50784d31fac400c531e462f648db7783b4d
.pes/objects/c2/dd5b0138fb03a0aa1bbd6e4bcff8b0f7667dbfd7812ca96c7ea93aaad68edd
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/refs/heads/main
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$
```

After three commits the object store contains multiple objects: blobs for each file version, tree objects for each commit snapshot, and commit objects themselves. The HEAD, index, and refs/heads/main reference files are also present.

Screenshot 4C — Reference Chain (refs/heads/main and HEAD)

Commands: cat .pes/refs/heads/main | cat .pes/HEAD

```
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ cat .pes/refs/heads/main
702451e82b7cfbb0df8abe8cef9b80c4442129c6f939a3cabc36a8cf086bced4
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$
```

refs/heads/main contains the full SHA-256 hash of the latest commit. HEAD contains the symbolic reference 'ref: refs/heads/main', confirming the standard attached-HEAD reference chain is correctly implemented.

Final Integration Test — make test-integration

Integration Test Output (Part 1)

Command: make test-integration

```
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 69193365096f... Initial commit

Log after first commit:
commit 69193365096f293b1630b399ac1a7d2ad2889365e75e06244921f82e743d48af
Author: YASHAS K C <PES2UG24CS669>
Date:   1776803493

    Initial commit


--- Second Commit ---
Committed: 64e87fbc9542... Update file.txt

--- Third Commit ---
Committed: 1ed51b43d835... Add farewell
```

Integration Test Output (Part 2 — Full History & Object Store)

```
--- Full History ---
commit 1ed51b43d835586d3a45924b69c3d2d28db26d4c120536a2b46bbff7210d59ab
Author: YASHAS K C <PES2UG24CS669>
Date:   1776803493

    Add farewell◆◆

commit 64e87fbc9542d738a48482f78e4a6e4d83ade651df69a8838da14eaaaeada3a9
Author: YASHAS K C <PES2UG24CS669>
Date:   1776803493

    Update file.txt

commit 69193365096f293b1630b399ac1a7d2ad2889365e75e06244921f82e743d48af
Author: YASHAS K C <PES2UG24CS669>
Date:   1776803493

    Initial committial cQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
1ed51b43d835586d3a45924b69c3d2d28db26d4c120536a2b46bbff7210d59ab

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/1e/d51b43d835586d3a45924b69c3d2d28db26d4c120536a2b46bbff7210d59ab
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/64/e87fbc9542d738a48482f78e4a6e4d83ade651df69a8838da14eaaaeada3a9
.pes/objects/69/193365096f293b1630b399ac1a7d2ad2889365e75e06244921f82e743d48af
.pes/objects/84/c5fed72c2dd9a3130e2e9b98033827f62793fb80d0295ccedf5fc7972d96eb
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/de/34fb483a5c16ce29ec3195dd505dfda7e35cc5fe16a99da4b2455b332a93cf
.pes/objects/ec/0af37b159ae92a5850ceaa6e497860729d6158aab2c6833b6068a1e3c5e8ef

=== All integration tests completed ===
yashas@ubuntu:~/PES2UG24CS669-pes-vcs$
```

All integration tests completed successfully. The test sequence verified repository initialization, staging of files, three sequential commits, correct log output, HEAD/reference chain consistency, and object store growth to 10 objects — covering all phases end-to-end.

Phase 5: Analysis — Branching and Checkout

Q5.1 — Implementing pes checkout <branch>

To implement 'pes checkout <branch>', three steps are required in .pes/:

1. Read the target branch file at .pes/refs/heads/<branch> to obtain the commit hash. If it does not exist, the command must abort with an error.

2. Walk the commit object to find its tree hash, then recursively expand the tree to enumerate all (path, blob-hash) pairs that represent the target snapshot.

3. For each file in the target tree, read the blob from the object store and write it to the working directory, creating directories as needed. Files tracked in the current HEAD tree but absent from the target tree must be deleted.

Finally, update .pes/HEAD to 'ref: refs/heads/<branch>' so it points to the new branch.

What makes this complex: the working directory must be updated atomically enough to avoid half-applied states. If a write fails mid-way, the repo is inconsistent. Additionally, the operation must

handle new files (write them), deleted files (remove them), and modified files (overwrite them) all in a single pass, which requires diffing two trees rather than simply applying one.

Q5.2 — Detecting Dirty Working Directory Conflicts

Without any external diff tool, a conflict can be detected using only the index and the object store as follows:

Step 1 — Identify files that differ between the current branch tree and the target branch tree by comparing their blob hashes.

Step 2 — For each such file, look it up in .pes/index. If the file's recorded mtime or size differs from the actual on-disk stat(), the working copy is potentially dirty. To confirm, re-hash the on-disk file and compare it with the index's stored blob hash.

If the on-disk hash differs from the index blob hash, the file has uncommitted local changes. If the index blob hash also differs from the target branch's blob hash for the same path, a three-way conflict exists: current tree != index != target tree. In this case checkout must refuse and report the conflicting path.

This approach uses only stat() metadata first for efficiency, falling back to full re-hashing only when metadata suggests a change — the same strategy Git uses.

Q5.3 — Detached HEAD State

In detached HEAD state, .pes/HEAD contains a raw commit hash instead of a symbolic reference like 'ref: refs/heads/main'. Commits made in this state are valid and fully stored in the object store — each new commit correctly records the previous commit hash as its parent. However, since no branch reference is being updated, these commits are reachable only via HEAD itself.

As soon as the user checks out a branch or any other reference, HEAD is updated to point elsewhere. The detached commits are now unreachable from any named reference. They will not appear in 'pes log' (which follows HEAD -> branch -> commit chain) and will eventually be deleted by garbage collection.

Recovery options: (a) If the user remembers any prefix of a dangling commit hash, they can run 'find .pes/objects -type f' to locate it, then create a new branch pointing to it: 'echo <hash> > .pes/refs/heads/recovery'. (b) Git maintains a reflog (.git/logs/HEAD) which records every HEAD movement, allowing 'git reflog' to show the lost commit hash. Implementing a similar .pes/logs/HEAD append-only log at commit time would make recovery trivial.

Phase 6: Analysis — Garbage Collection

Q6.1 — Finding and Deleting Unreachable Objects

Algorithm (Mark-and-Sweep):

Mark Phase — Build the set of all reachable hashes using a breadth-first or depth-first traversal starting from every named reference:

1. Enumerate all files under .pes/refs/ to collect the tip commit hash of every branch.

2. For each tip commit, add its hash to a reachable set (implemented as a hash set for O(1) lookup). Parse the commit object to find its tree hash and parent hash; recursively add both.

3. For each tree object encountered, parse its entries and add the hash of every child blob and sub-tree.

4. Continue until no new hashes are added.

Sweep Phase — Enumerate every file under .pes/objects/. Reconstruct each object's hash from its two-character directory prefix plus filename. If the hash is NOT in the reachable set, delete the file.

Data Structure: A hash set (e.g., C unordered_set or a boolean array indexed by a truncated hash) gives O(1) insert and lookup. For 100,000 commits on 50 branches, assume an average of 5 files per commit (1 commit object + 1 tree + ~3 blobs): that is roughly 500,000 objects to visit in the mark phase, plus another 500,000 file-system entries in the sweep phase — on the order of 1 million total operations, well within a few seconds.

Q6.2 — GC Race Condition with Concurrent Commits

Race condition scenario:

T=0: A commit operation hashes a new blob (hash X) and writes it to .pes/objects/xx/<rest>. The blob is now on disk but no commit or tree yet references it.

T=1: GC starts its mark phase. It traverses all reachable objects from all branch tips. Hash X is not reachable from any reference yet, so it is NOT added to the reachable set.

T=2: GC's sweep phase deletes hash X from the object store, since it appeared unreachable.

T=3: The commit operation finishes writing the tree and commit objects, which now reference hash X. The object store is corrupt: the commit points to a blob that no longer exists.

How Git avoids this: Git's GC uses a 'grace period' (default 2 weeks) — it never deletes objects newer than a configurable threshold (checked via file mtime). Any object written recently is assumed to belong to an in-progress operation and is left alone. Additionally, Git writes a gc.pid lock file and checks for other git processes before starting GC. For a production implementation, the safest approach is to use a two-phase protocol: (1) write all new objects first, (2) update references only after all objects are durable, so GC can never observe a partially-committed state that references missing objects.